

TASK15: #51619

1. install and check the functioning of server monitoring tools like

1.Nagios

2.Zabbix

2.Deeply study the commands used to monitor the server as well as the services

PART 1: INSTALL NAGIOS SERVER (RHEL / Amazon Linux)

Step 1 — Update the system

```
sudo dnf update -y
```

Makes sure your OS packages are current before installing anything new.

Step 2 — Install required software

```
sudo dnf install -y autoconf gcc glibc make wget unzip httpd php php-cli php-gd gd gd-  
devel openssl openssl-devel
```

Package	What it's for
gcc, autoconf, make	Tools to compile Nagios from source code
wget, unzip	Download and extract files
httpd	The web server — lets you view Nagios in a browser
php	Runs the web pages of the Nagios dashboard
gd, gd-devel	Used by Nagios to draw graphs/images
openssl	Handles secure connections

Step 3 — Start the web server

```
sudo systemctl enable httpd --now
```

Turns on Apache/httpd now, and makes sure it auto-starts on every reboot.

Step 4 — Create a dedicated Nagios user and group

```
sudo useradd nagios
```

```
sudo groupadd nagcmd
```

```
sudo usermod -a -G nagcmd nagios
```

```
sudo usermod -a -G nagcmd apache
```

Nagios runs as its own user for security. nagcmd is a special shared group — it lets the web server (Apache) send commands to Nagios (like "recheck this now") through a shared file.

Step 5 — Download and compile Nagios Core

```
cd /tmp
```

```
wget https://assets.nagios.com/downloads/nagioscore/releases/nagios-4.5.7.tar.gz
```

```
tar -xvzf nagios-4.5.7.tar.gz
```

```
cd nagios-4.5.7
```

```
./configure --with-nagios-group=nagios --with-command-group=nagcmd
```

```
make all
```

```
sudo make install
```

```
sudo make install-init
```

```
sudo make install-config
```

```
sudo make install-commandmode
```

```
sudo make install-webconf
```

This downloads the actual Nagios program, unpacks it, and builds/installs it — the engine that does all the monitoring. Each make install-* step installs a different piece, and you need all of them:

Command	Why you need it
make install	Installs the core Nagios program and plugins-runner files — the actual engine
make install-init	Installs the startup script so Nagios can be controlled with systemctl (start/stop/restart) and auto-starts on reboot
make install-config	Installs the default sample config files (hosts, services, contacts) — without this, Nagios has nothing to monitor and won't even start

Command	Why you need it
<code>make install-commandmode</code>	Sets up the command pipe file/folder (the one used for "Force Check", "Acknowledge", etc. from the web UI) with correct permissions
<code>make install-webconf</code>	Installs the Apache config file that tells the web server how to serve the Nagios web pages (URLs, CGI permissions, auth)

Skipping any one of these leaves a piece missing — e.g. skip `install-webconf` and the web UI won't load at all; skip `install-commandmode` and you'll hit the "Could not stat() command file" error we ran into earlier.

Step 6 — Set a web login password

```
sudo htpasswd -c /usr/local/nagios/etc/htpasswd.users nagiosadmin
```

Creates the username/password you'll use to log into the Nagios website. Type a password when prompted.

Step 7 — Install Nagios plugins

```
cd /tmp
```

```
wget https://nagios-plugins.org/download/nagios-plugins-2.4.6.tar.gz
```

```
tar -xvzf nagios-plugins-2.4.6.tar.gz
```

```
cd nagios-plugins-2.4.6
```

```
./configure --with-nagios-user=nagios --with-nagios-group=nagios
```

```
make
```

```
sudo make install
```

Plugins are the actual small programs that check things — CPU load, disk space, ping, etc. Nagios itself is just the engine; plugins do the real checking work.

Step 8 — Start Nagios and restart the web server

```
sudo systemctl enable nagios --now
```

```
sudo systemctl restart httpd
```

Turns on Nagios permanently and reloads Apache so it recognizes the new Nagios web pages.

Step 9 — Open the firewall for web access

```
sudo firewall-cmd --permanent --add-service=http
```

```
sudo firewall-cmd --reload
```

Allows browsers to reach the server on port 80 (normal web traffic).

Step 10 — Log in

Open a browser and go to:

`http://<your-server-ip>/nagios`

Username: nagiosadmin, password: what you set in Step 6.

PART 2: SET UP THE CLIENT (the machine you want to monitor)

Step 11 — Install EPEL (extra package repository)

```
sudo dnf install -y https://dl.fedoraproject.org/pub/epel/epel-release-latest-9.noarch.rpm
```

RHEL's default software list is limited. EPEL adds access to extra tools like NRPE that aren't included by default.

Step 12 — Install NRPE and plugins on the client

```
sudo dnf install -y nrpe nagios-plugins-load nagios-plugins-disk nagios-plugins-users nagios-plugins-procs
```

NRPE is a small agent that lets the Nagios server ask this client "how's your CPU/disk/etc doing?" The plugins are the actual check tools it runs locally.

Step 13 — Tell NRPE which server is allowed to ask it questions

```
sudo nano /etc/nagios/nrpe.cfg
```

Edit this line, adding your Nagios server's IP:

```
allowed_hosts=127.0.0.1,::1,<NAGIOS-SERVER-IP>
```

This is a security setting — only trusted IPs are allowed to query this client.

Step 14 — Define what checks NRPE can run

In the same file, add or fix these lines:

```
command[check_load]=/usr/lib64/nagios/plugins/check_load -w 2.0,1.5,1.0 -c 3.0,2.5,2.0
```

```
command[check_disk]=/usr/lib64/nagios/plugins/check_disk -w 20% -c 10% -p /
```

This tells NRPE exactly what to run and what numbers count as "Warning" (-w) or "Critical" (-c) when the server asks for check_load or check_disk.

Why these specific numbers? Load average thresholds should roughly match your CPU core count. Check your cores with:

```
nproc
```

If you have 2 cores, load above 2.0 means the CPU is overloaded — that's why we use 2.0,1.5,1.0 (warning) and 3.0,2.5,2.0 (critical) for a 2-core machine.

Step 15 — Start NRPE

```
sudo systemctl enable nrpe --now
```

Turns on the NRPE agent so it starts listening for requests from the server.

Step 16 — Open the firewall port NRPE uses

```
sudo firewall-cmd --permanent --add-port=5666/tcp
```

```
sudo firewall-cmd --reload
```

NRPE communicates over port 5666 — this allows that traffic through.

Step 17 — Also open port 5666 in AWS Security Group

In the AWS Console: go to this instance → Security tab → Security Group → Inbound Rules → Add Rule → Custom TCP, Port 5666, Source = your Nagios server's IP. AWS has its own separate firewall (Security Groups) on top of the OS firewall — both need to allow the connection.

PART 3: CONNECT THE SERVER TO THE CLIENT

Step 18 — Install the plugin that lets the SERVER talk to NRPE

```
sudo dnf install -y nagios-plugins-nrpe
```

This installs check_nrpe — the tool the Nagios server uses to actually contact the client's NRPE agent.

Step 19 — Fix the plugin path if needed

```
sudo ln -s /usr/lib64/nagios/plugins/check_nrpe /usr/local/nagios/libexec/check_nrpe
```

Sometimes the plugin installs to a different folder than Nagios expects. This creates a shortcut (symlink) so Nagios can find it in the folder it actually looks in.

Step 20 — Test the connection

```
/usr/local/nagios/libexec/check_nrpe -H <CLIENT-IP>
```

This manually asks the client "are you there?" — a working response confirms the network path and NRPE agent are both fine before we configure anything else.

Step 21 — Tell Nagios how to run check_nrpe (define the command)

```
sudo nano /usr/local/nagios/etc/objects/commands.cfg
```

Add at the bottom:

```
define command{  
    command_name    check_nrpe  
    command_line    $USER1$/check_nrpe -H $HOSTADDRESS$ -c $ARG1$  
}
```

Nagios needs every check "command" formally defined somewhere before it can be used in a service — this tells Nagios exactly what check_nrpe means and how to run it.

Step 22 — Define the client as a host, and its services to check

```
sudo nano /usr/local/nagios/etc/objects/client1.cfg
```

Paste in:

```
define host{  
    use                linux-server  
    host_name          client1  
    alias              Client Server  
    address            <CLIENT-IP>  
    max_check_attempts 5  
    check_period       24x7  
    notification_interval 30  
    notification_period 24x7  
}
```

```
define service{  
    use                generic-service
```

```
host_name      client1

service_description  Current Load

check_command    check_nrpe!check_load
}
```

```
define service{

    use          generic-service

    host_name      client1

    service_description  Root Partition

    check_command    check_nrpe!check_disk
}
```

This is the actual "add a new monitored machine" step — it tells Nagios: here's a new host called client1, at this IP, and here are the two things to check on it (Load and Disk), using the check_nrpe command we just defined.

Step 23 — Tell Nagios to load this new config file

```
sudo nano /usr/local/nagios/etc/nagios.cfg
```

Add this line among the others like it:

```
cfg_file=/usr/local/nagios/etc/objects/client1.cfg
```

Nagios only reads config files it's explicitly told about — this line registers your new file.

Step 24 — Check for mistakes before restarting

```
sudo /usr/local/nagios/bin/nagios -v /usr/local/nagios/etc/nagios.cfg
```

This is a "dry run" — it checks your config files for errors without actually restarting anything. Look for Total Errors: 0 at the bottom.

Step 25 — Restart Nagios to apply everything

```
sudo systemctl restart nagios
```

Step 26 — Check the web UI

Go to <http://<server-ip>/nagios> → click **Hosts** — you should now see both localhost and client1 listed.

PART 4: LOAD TESTING (to see monitoring actually react)

Step 27 — Check current CPU load

uptime

Shows load average over the last 1, 5, and 15 minutes.

Step 28 — Check your CPU core count

nproc

Tells you how many cores you have, so you know what "normal" vs "overloaded" load looks like.

Step 29 — Install a load-generating tool

```
sudo dnf install -y stress-ng
```

(if this fails, install EPEL first — Step 11)

Step 30 — Generate CPU load

```
stress-ng --cpu 2 --timeout 60s
```

Runs 2 stress workers (match this to your actual core count) for 60 seconds, artificially spiking your CPU load so you can watch monitoring react.

Step 31 — Watch the load rise live

```
watch -n 1 uptime
```

Refreshes the load numbers every second so you can see them climb in real time.

Step 32 — Stop the load test

```
pkill stress-ng
```

Kills the stress-ng process immediately, letting the system return to normal.

Step 33 — Confirm it's actually stopped

```
pgrep stress-ng
```

No output = nothing running. Any number shown = still running.

Step 34 — Force Nagios to check right now (don't wait for auto-schedule)

In the web UI: click the service name (e.g. "Current Load") → "Re-schedule the next check of this service" → check "Force Check" → Commit. Nagios normally checks on a timer (every few minutes) — this makes it check immediately so you can see results without waiting.

PART 5: TROUBLESHOOTING COMMANDS (used when something's wrong)

Check if a service is running

```
sudo systemctl status nagios
```

```
sudo systemctl status nrpe
```

```
sudo systemctl status httpd
```

Shows whether each program is active, and shows recent log lines if it crashed.

Check NRPE's live activity log

```
sudo journalctl -u nrpe -f
```

Shows what NRPE is doing in real-time — useful to catch exactly why a connection was rejected.

Check if a port is being listened on

```
sudo ss -tulpn | grep 5666
```

Confirms NRPE is actually listening for connections on its port.

Check firewall rules

```
sudo firewall-cmd --list-ports
```

```
sudo firewall-cmd --list-services
```

Shows what's currently allowed through the OS firewall.

Manually run a check (bypass Nagios entirely)

```
/usr/local/nagios/libexec/check_load -w 2.0,1.5,1.0 -c 3.0,2.5,2.0
```

```
/usr/local/nagios/libexec/check_nrpe -H <CLIENT-IP> -c check_load
```

Runs the exact same check Nagios would run, showing you the raw, real result — helpful for confirming whether a problem is in the check itself or somewhere else (like the web UI display).

Find a missing file/program anywhere on the system

```
sudo find / -name "check_nrpe" 2>/dev/null
```

Searches your whole filesystem — useful when a package installs somewhere unexpected.

Check what package actually got installed

```
rpm -qa | grep nrpe
```

Lists installed packages matching a keyword, confirming whether something actually installed.

PART 6: FIXING COMMON ISSUES

Fix: "Could not stat() command file" error in web UI

```
groups apache
```

```
sudo usermod -a -G nagcmd apache
```

```
sudo systemctl restart httpd
```

Means Apache can't send commands to Nagios. Fix is making sure the Apache user is in the nagcmd group, then restarting Apache so the change applies.

Fix: "Command not defined" NRPE error

Add the missing command to /etc/nagios/nrpe.cfg on the **client**, then:

```
sudo systemctl restart nrpe
```

Every check the server asks for must be individually defined on the client's NRPE config — restarting applies any edits.

Fix: No swap space (Swap Usage shows CRITICAL)

```
sudo dd if=/dev/zero of=/swapfile bs=1M count=1024
```

```
sudo chmod 600 /swapfile
```

```
sudo mkswap /swapfile
```

```
sudo swapon /swapfile
```

```
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
```

Creates real swap space (extra "overflow" memory on disk) so the check has something real to measure, and makes it permanent across reboots.

QUICK REFERENCE — DAILY USE COMMANDS

```
sudo systemctl restart nagios    # apply any config changes
```

```
sudo systemctl restart nrpe      # apply any client-side NRPE changes
```

```
sudo systemctl restart httpd     # apply any web server changes
```

sudo /usr/local/nagios/bin/nagios -v /usr/local/nagios/etc/nagios.cfg # ALWAYS run this before restarting nagios, to catch config mistakes first

uptime # check load average

nproc # check CPU core count

stress-ng --cpu 2 --timeout 60s # generate test load

pkill stress-ng # stop test load

OUTPUTS: MAIN SERVER

Task #51619: Ser...

server monitori...

nagios monitori...

Free Cloud Com...

Instances | EC2 |

Connect to inst...

Nagios: 13.203.2...

Ask Gemini

13.203.214.120/nagios/

Practice

All Bookmarks

Nagios

General

Home

Documentation

Current Status

Tactical Overview

Map

Hosts

Services

Host Groups

Summary

Grid

Service Groups

Summary

Grid

Problems

Services (Unhandled)

Hosts (Unhandled)

Network Outages

Quick Search:

Reports

Availability

Trends

Alerts

History

Summary

Histogram

Notifications

Event Log

System

Current Network Status

Last Updated: Tue Sep 22 05:49:40 UTC 2026

Updated every 90 seconds

Nagios® Core™ 4.5.7 - www.nagios.org

Logged in as nagiosadmin

View History For all hosts

View Notifications For All Hosts

View Host Status Detail For All Hosts

Host Status Totals

Up Down Unreachable Pending

1 0 0 0

All Problems All Types

0 1

Service Status Totals

Ok Warning Unknown Critical Pending

6 1 0 1 0

All Problems All Types

2 8

Service Status Details For All Hosts

Limit Results: 100

Host	Service	Status	Last Check	Duration	Attempt	Status Information
localhost	Current Load	OK	09-22-2026 05:45:40	0d 0h 44m 0s	1/4	OK - load average: 3.42, 1.48, 1.18
	Current Users	OK	09-22-2026 05:46:16	0d 0h 43m 22s	1/4	USERS OK - 3 users currently logged in
	HTTP	WARNING	09-22-2026 05:44:55	0d 0h 39m 45s	4/4	HTTP WARNING: HTTP/1.1 403 Forbidden - 6199 bytes in 0.001 second response time
	PING	OK	09-22-2026 05:47:33	0d 0h 42m 7s	1/4	PING OK - Packet loss = 0%, RTT = 0.05 ms
	Root Partition	OK	09-22-2026 05:48:10	0d 0h 41m 30s	1/4	DISK OK - free space: / 7382 MB (74.01% inode=99%):
	SSH	OK	09-22-2026 05:48:48	0d 0h 40m 52s	1/4	SSH OK - OpenSSH_8.9 (protocol 2.0)
	Swap Usage	CRITICAL	09-22-2026 05:47:25	0d 0h 37m 15s	4/4	SWAP CRITICAL - 0% free (0 MB out of 0 MB) - Swap is either disabled, not present, or of zero size.
	Total Processes	OK	09-22-2026 05:45:03	0d 0h 39m 37s	1/4	PROCS OK: 40 processes with STATE = RSDZT

Results 1 - 8 of 8 Matching Services

Task #51619: Ser...

server monitori...

nagios monitori...

Free Cloud Com...

Instances | EC2 |

Connect to inst...

Connect to inst...

Nagios: 13.203.2...

Ask Gemini

13.203.214.120/nagios/

Practice

All Bookmarks

Nagios

General

Home

Documentation

Current Status

Tactical Overview

Map

Hosts

Services

Host Groups

Summary

Grid

Service Groups

Summary

Grid

Problems

Services (Unhandled)

Hosts (Unhandled)

Network Outages

Quick Search:

Reports

Availability

Trends

Alerts

History

Summary

Histogram

Notifications

Event Log

System

Current Network Status

Last Updated: Tue Sep 22 06:24:19 UTC 2026

Updated every 90 seconds

Nagios® Core™ 4.5.7 - www.nagios.org

Logged in as nagiosadmin

View History For all hosts

View Notifications For All Hosts

View Host Status Detail For All Hosts

Host Status Totals

Up Down Unreachable Pending

1 0 0 0

All Problems All Types

0 1

Service Status Totals

Ok Warning Unknown Critical Pending

6 1 0 2 0

All Problems All Types

3 8

Service Status Details For All Hosts

Limit Results: 100

Host	Service	Status	Last Check	Duration	Attempt	Status Information
localhost	Current Load	CRITICAL	09-22-2026 06:21:40	0d 0h 17m 39s	4/4	CRITICAL - load average: 19.35, 24.83, 19.82
	Current Users	OK	09-22-2026 06:21:18	0d 1h 18m 1s	1/4	USERS OK - 4 users currently logged in
	HTTP	WARNING	09-22-2026 06:19:55	0d 1h 14m 24s	4/4	HTTP WARNING: HTTP/1.1 403 Forbidden - 6199 bytes in 0.001 second response time
	PING	OK	09-22-2026 06:22:33	0d 1h 16m 46s	1/4	PING OK - Packet loss = 0%, RTT = 0.05 ms
	Root Partition	OK	09-22-2026 06:23:10	0d 1h 16m 9s	1/4	DISK OK - free space: / 7382 MB (74.01% inode=99%):
	SSH	OK	09-22-2026 06:23:48	0d 1h 15m 31s	1/4	SSH OK - OpenSSH_8.9 (protocol 2.0)
	Swap Usage	CRITICAL	09-22-2026 06:22:25	0d 1h 11m 54s	4/4	SWAP CRITICAL - 0% free (0 MB out of 0 MB) - Swap is either disabled, not present, or of zero size.
	Total Processes	OK	09-22-2026 06:20:03	0d 1h 14m 16s	1/4	PROCS OK: 40 processes with STATE = RSDZT

Results 1 - 8 of 8 Matching Services

CLIENT SERVER :

Nagios

Current Network Status
 Last Updated: Tue Sep 22 07:12:41 UTC 2026
 Updated every 90 seconds
 Nagios® Core™ 4.5.7 - www.nagios.org
 Logged in as nagiosadmin

Host Status Totals
 Up: 2, Down: 0, Unreachable: 0, Pending: 0
 All Problems: 0, All Types: 2

Service Status Totals
 OK: 6, Warning: 1, Unknown: 1, Critical: 2, Pending: 0
 All Problems: 4, All Types: 10

Service Status Details For All Hosts

Host	Service	Status	Last Check	Duration	Attempt	Status Information
client1	Current Load	CRITICAL	09-22-2026 07:11:36	0d 0h 3m 3s	2/3	CRITICAL - load average per CPU: 0.20, 2.17, 1.71
	Root Partition	UNKNOWN	09-22-2026 07:04:50	0d 0h 7m 51s	3/3	NRPE: Command 'check_disk' not defined
localhost	Current Load	OK	09-22-2026 07:11:40	0d 0h 21m 1s	1/4	OK - load average: 0.00, 0.00, 0.75
	Current Users	OK	09-22-2026 07:11:18	0d 2h 6m 23s	1/4	USERS OK - 3 users currently logged in
	HTTP	WARNING	09-22-2026 07:09:55	0d 2h 2m 46s	4/4	HTTP WARNING: HTTP/1.1 403 Forbidden - 6199 bytes in 0.001 second response time
	PING	OK	09-22-2026 07:07:33	0d 2h 5m 6s	1/4	PING OK - Packet loss = 0%, RTT = 0.06 ms
	Root Partition	OK	09-22-2026 07:06:10	0d 2h 4m 31s	1/4	DISK OK - free space: / 7345 MB (73.64% inode=99%):
	SSH	OK	09-22-2026 07:06:48	0d 2h 3m 53s	1/4	SSH OK - OpenSSH_9.9 (protocol 2.0)
	Swap Usage	CRITICAL	09-22-2026 07:12:25	0d 2h 0m 16s	4/4	SWAP CRITICAL - 0% free IO MB out of 0 MB! - Swap is either disabled, not present, or of zero size.
	Total Processes	OK	09-22-2026 07:10:03	0d 2h 2m 38s	1/4	PROCS OK: 40 processes with STATE = RSZDT

Results 1 - 10 of 10 Matching Services

Nagios

Current Network Status
 Last Updated: Tue Sep 22 07:02:25 UTC 2026
 Updated every 90 seconds
 Nagios® Core™ 4.5.7 - www.nagios.org
 Logged in as nagiosadmin

Host Status Totals
 Up: 2, Down: 0, Unreachable: 0, Pending: 0
 All Problems: 0, All Types: 2

Service Status Totals
 OK: 7, Warning: 1, Unknown: 1, Critical: 1, Pending: 0
 All Problems: 3, All Types: 10

Service Status Details For All Hosts

Host	Service	Status	Last Check	Duration	Attempt	Status Information
client1	Current Load	OK	09-22-2026 06:59:38	0d 0h 3m 59s+	1/3	OK - load average per CPU: 0.00, 0.00, 0.00
	Root Partition	UNKNOWN	09-22-2026 07:00:50	0d 0h 1m 35s	1/3	NRPE: Command 'check_disk' not defined
localhost	Current Load	OK	09-22-2026 07:01:40	0d 0h 10m 45s	1/4	OK - load average: 0.00, 0.00, 1.46
	Current Users	OK	09-22-2026 07:01:18	0d 1h 56m 7s	1/4	USERS OK - 3 users currently logged in
	HTTP	WARNING	09-22-2026 06:59:55	0d 1h 52m 30s	4/4	HTTP WARNING: HTTP/1.1 403 Forbidden - 6199 bytes in 0.001 second response time
	PING	OK	09-22-2026 06:57:33	0d 1h 54m 52s	1/4	PING OK - Packet loss = 0%, RTT = 0.05 ms
	Root Partition	OK	09-22-2026 06:58:10	0d 1h 54m 15s	1/4	DISK OK - free space: / 7345 MB (73.64% inode=99%):
	SSH	OK	09-22-2026 06:58:48	0d 1h 53m 37s	1/4	SSH OK - OpenSSH_9.9 (protocol 2.0)
	Swap Usage	CRITICAL	09-22-2026 06:57:25	0d 1h 50m 0s	4/4	SWAP CRITICAL - 0% free IO MB out of 0 MB! - Swap is either disabled, not present, or of zero size.
	Total Processes	OK	09-22-2026 07:00:03	0d 1h 52m 22s	1/4	PROCS OK: 40 processes with STATE = RSZDT

Results 1 - 10 of 10 Matching Services

PART 1: INSTALL ZABBIX SERVER (RHEL / Amazon Linux)

Step 1 — Add the official Zabbix repository

```
bash
```

```
sudo rpm -Uvh https://repo.zabbix.com/zabbix/7.0/rhel/9/x86_64/zabbix-release-7.0-2.el9.noarch.rpm
```

```
sudo dnf clean all
```

RHEL doesn't include Zabbix packages by default — this adds Zabbix's own repo so dnf knows where to find them. Use /rhel/8/ in the URL if you're on RHEL 8.

Step 2 — Install Zabbix server, web frontend, and agent

```
bash
```

```
sudo dnf install -y zabbix-server-mysql zabbix-web-mysql zabbix-nginx-conf zabbix-sql-  
scripts zabbix-agent --disablerepo=epel
```

--disablerepo=epel is important — EPEL also ships its own (older, conflicting) Zabbix packages, so this forces dnf to use only the official Zabbix repo.

Step 3 — Install and start MariaDB (the database Zabbix needs)

```
bash
```

```
sudo dnf install -y mariadb-server
```

```
sudo systemctl enable mariadb --now
```

Zabbix stores all its config, history, and alert data in a MySQL/MariaDB database — unlike Nagios, which uses flat config files.

Step 4 — Secure MariaDB

```
bash
```

```
sudo mysql_secure_installation
```

Sets a root password and removes insecure defaults.

Step 5 — Create the Zabbix database and user

```
bash
```

```
sudo mysql -uroot
```

Note: on RHEL/MariaDB, root usually authenticates via sudo only — no -p flag needed. Then inside the MySQL prompt:

```
sql
```

```
CREATE DATABASE zabbix character set utf8mb4 collate utf8mb4_bin;
```

```
CREATE USER zabbix@localhost IDENTIFIED BY 'YourPassword';
```

```
GRANT ALL PRIVILEGES ON zabbix.* TO zabbix@localhost;
```

```
SET GLOBAL log_bin_trust_function_creators = 1;
```

```
QUIT;
```

This creates a dedicated database and user just for Zabbix, with access limited to only that database.

Step 6 — Import Zabbix's table structure (run this ONCE only)

```
bash
```

```
zcat /usr/share/zabbix-sql-scripts/mysql/server.sql.gz | mysql --default-character-set=utf8mb4 -uzabbix -p zabbix
```

Loads all the empty tables Zabbix needs, including default data like the built-in Admin user. **Do not run this twice** — running it again after it's partially/fully succeeded causes "table already exists" errors and can leave critical tables (like users) empty, which stops Zabbix server from starting.

Step 7 — Set the database password in Zabbix's config

```
bash
```

```
sudo nano /etc/zabbix/zabbix_server.conf
```

Set:

```
DBPassword=YourPassword
```

PART 2: FIX WEB SERVER CONFLICT (using httpd instead of nginx)

Since Nagios already uses Apache/httpd on port 80, we swapped Zabbix's web frontend from Nginx to Apache to avoid a port conflict.

Step 8 — Remove nginx, install the Apache config package instead

```
bash
```

```
sudo systemctl stop nginx
```

```
sudo systemctl disable nginx
```

```
sudo dnf remove -y zabbix-nginx-conf
```

```
sudo dnf install -y zabbix-apache-conf --disablerepo=epel
```

Step 9 — Set PHP timezone (required by Zabbix's web UI)

```
bash
```

```
sudo nano /etc/php.ini
```

Find ;date.timezone = and change to:

```
date.timezone = Asia/Kolkata
```

(Uncomment it — remove the leading ; — and set your actual timezone.)

Step 10 — Start httpd and php-fpm

```
bash
```

```
sudo systemctl restart httpd php-fpm
```

```
sudo systemctl status httpd
```

Since the zabbix-apache-conf package uses an **Alias** (/zabbix) rather than a separate port, Zabbix ends up accessible at `http://<server-ip>/zabbix` — same port 80 as Nagios, different URL path, no conflict.

PART 3: START ZABBIX SERVER AND FINISH WEB SETUP

Step 11 — Start Zabbix server and agent

```
bash
```

```
sudo systemctl restart zabbix-server zabbix-agent httpd php-fpm
```

```
sudo systemctl enable zabbix-server zabbix-agent httpd php-fpm
```

Step 12 — Open port 80 in the firewall (if not already open for Nagios)

```
bash
```

```
sudo firewall-cmd --permanent --add-service=http
```

```
sudo firewall-cmd --reload
```

Step 13 — Access the setup wizard

```
http://<server-ip>/zabbix
```

Walk through: Database type (MySQL), host (localhost), name (zabbix), user (zabbix), password (from Step 5) → Next → Install → Finish.

Step 14 — Log in

Username: Admin

Password: zabbix

(default login — change immediately after first login)

PART 4: SET UP THE CLIENT (the machine you want to monitor)

Step 15 — On the CLIENT, add the Zabbix repo

```
bash
```

```
sudo rpm -Uvh https://repo.zabbix.com/zabbix/7.0/rhel/9/x86_64/zabbix-release-7.0-2.el9.noarch.rpm
```

```
sudo dnf clean all
```

Step 16 — Install just the agent (client doesn't need server/database/web)

```
bash
```

```
sudo dnf install -y zabbix-agent --disablerepo=epel
```

Step 17 — Configure the agent

```
bash
```

```
sudo nano /etc/zabbix/zabbix_agentd.conf
```

Set these three lines exactly:

```
Server=<ZABBIX-SERVER-IP>
```

```
ServerActive=<ZABBIX-SERVER-IP>
```

```
Hostname=client1
```

Setting	What it does
----------------	---------------------

Server=	Which IP is allowed to query this agent (like an allow-list)
---------	--

ServerActive=	Which server this agent actively sends data to
---------------	--

Hostname=	Must exactly match the host name you create in the Zabbix web UI
-----------	--

Step 18 — Start the agent

```
bash
```

```
sudo systemctl enable zabbix-agent --now
```

```
sudo systemctl status zabbix-agent
```

Step 19 — Open the firewall port (10050)

```
bash
```

```
sudo firewall-cmd --permanent --add-port=10050/tcp
```

```
sudo firewall-cmd --reload
```

Step 20 — Open port 10050 in AWS Security Group

AWS Console → client instance → Security tab → Security Group → Inbound Rules → Add:

- Type: Custom TCP, Port: 10050, Source: your Zabbix server's IP

PART 5: CONNECT SERVER TO CLIENT (in the web UI)

Step 21 — Create the host in Zabbix

Web UI → **Data collection** → **Hosts** → **Create host**

- Host name: client1 (must match agent config exactly)
- Host groups: e.g. "Linux servers"
- Interfaces → Agent → enter client's IP, port 10050
- Templates → Select → search "Linux by Zabbix agent" → Add

Click **Add** to save.

Step 22 — Check connection status

Go to **Data collection** → **Hosts** — check the **Availability (ZBX)** column:

- Green = connected and working
- Red/grey = not reachable yet (see troubleshooting below)

PART 6: LOAD TESTING

Step 23 — Check baseline

bash

uptime

nproc

Step 24 — Generate load

bash

stress-ng --cpu 2 --timeout 60s

Step 25 — View it in Zabbix

Web UI → **Monitoring** → **Latest data** → filter Host = your host → find "Load average (1m avg)" etc. → click **Graph**. Note: Zabbix updates on its normal interval (often ~1 minute) — not instant like a forced Nagios check, so allow time before refreshing.

Step 26 — Stop the load test

bash

pkill stress-ng

PART 7: TROUBLESHOOTING COMMANDS

Test if the server can reach the client's agent (like check_nrpe, but for Zabbix)

bash

```
sudo dnf install -y zabbix-get --disablerepo=epel # install the tool if missing, on the SERVER
```

```
zabbix_get -s <CLIENT-IP> -k agent.ping
```

A successful response returns 1.

Test raw network/port connectivity (bypasses Zabbix entirely)

bash

```
timeout 5 bash -c "</dev/tcp/<CLIENT-IP>/10050" && echo "Port is OPEN" || echo "Port is CLOSED/BLOCKED"
```

This is the fastest way to tell if it's a firewall/network problem vs. a Zabbix config problem.

Basic reachability test

bash

```
ping -c 4 <CLIENT-IP>
```

Check if the agent is running and listening (on the CLIENT)

bash

```
sudo systemctl status zabbix-agent
```

```
sudo ss -tulpn | grep 10050
```

Check agent config values (on the CLIENT)

bash

```
grep -E "^Server=|^ServerActive=|^Hostname=" /etc/zabbix/zabbix_agentd.conf
```

Check local firewall (on the CLIENT)

bash

```
sudo firewall-cmd --list-ports
```

Check Zabbix server logs (on the SERVER)

bash

```
sudo tail -40 /var/log/zabbix/zabbix_server.log
```

Check what package/binary actually installed, and where

```
bash
```

```
rpm -qa | grep zabbix-get
```

```
which zabbix_get
```

```
sudo find / -name "zabbix_get" 2>/dev/null
```

Change the zabbix MySQL user's password

```
bash
```

```
sudo mysql -uroot
```

```
sql
```

```
ALTER USER 'zabbix'@'localhost' IDENTIFIED BY 'NewPassword';
```

```
FLUSH PRIVILEGES;
```

```
QUIT;
```

Then update /etc/zabbix/zabbix_server.conf (DBPassword=) and restart:

```
bash
```

```
sudo systemctl restart zabbix-server
```

If zabbix-server won't start due to a corrupted/partial database import

```
bash
```

```
sudo mysql -uroot -e "DROP DATABASE zabbix;"
```

```
sudo mysql -uroot -e "CREATE DATABASE zabbix character set utf8mb4 collate  
utf8mb4_bin;"
```

```
sudo mysql -uroot -e "GRANT ALL PRIVILEGES ON zabbix.* TO zabbix@localhost;"
```

```
sudo mysql -uroot -e "FLUSH PRIVILEGES;"
```

```
zcat /usr/share/zabbix-sql-scripts/mysql/server.sql.gz | mysql --default-character-  
set=utf8mb4 -uzabbix -p zabbix
```

Then verify:

```
bash
```

```
mysql -uzabbix -p zabbix -e "SELECT COUNT(*) FROM users;"
```

OUTPUTS: MAIN SERVER

The screenshot displays the Zabbix web interface for configuring and monitoring a 'load' tag. The interface is divided into a sidebar and a main content area.

Sidebar: Contains navigation links for Dashboards, Monitoring, Problems, Hosts, Latest data, Maps, Discovery, Services, Inventory, Reports, Data collection, Alerts, Users, Administration, Support, and Integrations.

Main Content Area:

- Name:** load
- Show tags:** None, 1, 2, 3 (selected)
- Tag name:** Full, Shortened, None
- Tag display priority:** comma-separated list
- State:** All, Normal, Not supported
- Show details:** ☐
- Buttons:** Save as, Apply, Reset
- Subfilter affects only filtered data**
- HOSTS:** Zabbix server 3
- TAGS:** component 3
- TAG VALUES:** component: cpu 3
- DATA:** With data, Without data

Table of Monitored Items:

☐	Host	Name	Last check	Last value	Change	Tags	Info
☐	Zabbix server	Load average (1m avg)	54s	12.335	+1.1523	component: cpu	Graph
☐	Zabbix server	Load average (5m avg)	46s	6.7705	+1.3901	component: cpu	Graph
☐	Zabbix server	Load average (15m avg)	44s	3.1719	+0.6592	component: cpu	Graph

Displaying 3 of 3 found

Zabbix 7.0.30. © 2001–2026, Zabbix SIA